

```
===== Python Code =====
import numpy as np

# Step 1: Create matrices A and B
A = np.array([
    [1, 2, -10, 4],
    [3, 4, 5, -6],
    [3, 3, -2, 5]
])

B = np.array([3, 3, 4, 2])

print("A =")
print(A)
print("\nB =")
print(B)

# Step 2: Find lengths
lengthA = max(A.shape)
lengthB = len(B)

print("\nlengthA =", lengthA)
print("lengthB =", lengthB)

# Step 3: Add B as the fourth row of A to create C
C = np.vstack((A, B))

print("\nC =")
print(C)

# Step 4: Extract rows 2 to 4 and columns 3 to 4 from C to create D
D = C[1:4, 2:4]

print("\nD =")
print(D)

# Step 5: Transpose D to create E
E = D.T

print("\nE =")
print(E)

# Step 6: Find the size of E
m, n = E.shape

print("\nm =", m)
print("n =", n)

# Step 7: Create equally spaced values from 0 to 2*pi
x = np.linspace(0, 2*np.pi, 50)

print("\nx =")
print(x)

# Step 8: Column max and min of A
col_max = np.max(A, axis=0)
col_min = np.min(A, axis=0)

print("\nColumn max =", col_max)
print("Column min =", col_min)

# Step 9: Row max and min, overall max and min
row_max = np.max(A, axis=1)
row_min = np.min(A, axis=1)
overall_max = np.max(A)
overall_min = np.min(A)

print("\nRow max =", row_max)
```

```
print("Row min =", row_min)
print("Overall max =", overall_max)
print("Overall min =", overall_min)

# Step 10: Means and sums
meancolA = np.mean(A, axis=0)
meanrowA = np.mean(A, axis=1)
sumcolA = np.sum(A, axis=0)
sumrowA = np.sum(A, axis=1)
meanA = np.mean(A)
sumA = np.sum(A)

print("\nColumn mean =", meancolA)
print("Row mean =", meanrowA)
print("Column sum =", sumcolA)
print("Row sum =", sumrowA)
print("Overall mean =", meanA)
print("Overall sum =", sumA)

# Step 11: Random integer matrices F and G, size 5x3, values from -4 to 4
F = np.random.randint(-4, 5, (5, 3))
G = np.random.randint(-4, 5, (5, 3))

print("\nF =")
print(F)
print("\nG =")
print(G)

# Step 12: Operations on F and G
ScMultF = 0.4 * F
SumFG = F + G
DiffFG = F - G
ElProdFG = F * G

print("\n0.4 * F =")
print(ScMultF)
print("\nF + G =")
print(SumFG)
print("\nF - G =")
print(DiffFG)
print("\nElementwise F * G =")
print(ElProdFG)

# Step 13: Sizes of F and A
sizeF = F.shape
sizeA = A.shape

print("\nsizeF =", sizeF)
print("sizeA =", sizeA)

# Step 14: Matrix multiplication H = F * A
H = F @ A

print("\nH =")
print(H)

# Step 15: Identity matrix
I = np.eye(3)

print("\nI =")
print(I)

# Step 16: Zero and one matrices
Z = np.zeros((3, 4))
O = np.ones((2, 4))

print("\nZ =")
print(Z)
```

```
print("\nO =")
print(O)

# Step 17: Diagonal matrix with entries 1, 2, 7
D_diag = np.diag([1, 2, 7])

print("\nD_diag =")
print(D_diag)

# Step 18: Random 6x6 matrix and its diagonal entries
M = np.random.randint(1, 10, (6, 6))
diag_entries = np.diag(M)

print("\nM =")
print(M)
print("\nDiagonal entries =")
print(diag_entries)

# Step 19: Tridiagonal matrix
n = 6
main_diag = 2 * np.ones(n)
upper_diag = -1 * np.ones(n - 1)
lower_diag = -1 * np.ones(n - 1)

T = np.diag(main_diag) + np.diag(upper_diag, 1) + np.diag(lower_diag, -1)

print("\nT =")
print(T)
```

===== Program Output =====

```
A =
[[ 1  2 -10  4]
 [ 3  4  5 -6]
 [ 3  3 -2  5]]

B =
[3 3 4 2]

lengthA = 4
lengthB = 4

C =
[[ 1  2 -10  4]
 [ 3  4  5 -6]
 [ 3  3 -2  5]
 [ 3  3  4  2]]

D =
[[ 5 -6]
 [-2  5]
 [ 4  2]]

E =
[[ 5 -2  4]
 [-6  5  2]]

m = 2
n = 3

x =
[0.          0.12822827  0.25645654  0.38468481  0.51291309  0.64114136
 0.76936963  0.8975979  1.02582617  1.15405444  1.28228272  1.41051099
 1.53873926  1.66696753  1.7951958  1.92342407  2.05165235  2.17988062
 2.30810889  2.43633716  2.56456543  2.6927937  2.82102197  2.94925025
 3.07747852  3.20570679  3.33393506  3.46216333  3.5903916  3.71861988
 3.84684815  3.97507642  4.10330469  4.23153296  4.35976123  4.48798951
 4.61621778  4.74444605  4.87267432  5.00090259  5.12913086  5.25735913
 5.38558741  5.51381568  5.64204395  5.77027222  5.89850049  6.02672876]
```

```
6.15495704 6.28318531]
```

```
Column max = [3 4 5 5]
```

```
Column min = [ 1 2 -10 -6]
```

```
Row max = [4 5 5]
```

```
Row min = [-10 -6 -2]
```

```
Overall max = 5
```

```
Overall min = -10
```

```
Column mean = [ 2.33333333 3. -2.33333333 1. ]
```

```
Row mean = [-0.75 1.5 2.25]
```

```
Column sum = [ 7 9 -7 3]
```

```
Row sum = [-3 6 9]
```

```
Overall mean = 1.0
```

```
Overall sum = 12
```

```
F =
```

```
[[ 0 0 -4]
 [ 4 3 0]
 [ 0 -2 -2]
 [ 1 -2 -1]
 [ 3 0 -1]]
```

```
G =
```

```
[[ 3 -1 0]
 [ 0 4 4]
 [-2 4 3]
 [-4 0 4]
 [ 3 2 3]]
```

```
0.4 * F =
```

```
[[ 0. 0. -1.6]
 [ 1.6 1.2 0. ]
 [ 0. -0.8 -0.8]
 [ 0.4 -0.8 -0.4]
 [ 1.2 0. -0.4]]
```

```
F + G =
```

```
[[ 3 -1 -4]
 [ 4 7 4]
 [-2 2 1]
 [-3 -2 3]
 [ 6 2 2]]
```

```
F - G =
```

```
[[ -3 1 -4]
 [ 4 -1 -4]
 [ 2 -6 -5]
 [ 5 -2 -5]
 [ 0 -2 -4]]
```

```
Elementwise F * G =
```

```
[[ 0 0 0]
 [ 0 12 0]
 [ 0 -8 -6]
 [-4 0 -4]
 [ 9 0 -3]]
```

```
sizeF = (5, 3)
```

```
sizeA = (3, 4)
```

```
H =
```

```
[[ -12 -12 8 -20]
 [ 13 20 -25 -2]
 [-12 -14 -6 2]
 [ -8 -9 -18 11]
 [ 0 3 -28 7]]
```

```
I =  
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]
```

```
Z =  
[[0. 0. 0. 0.]  
 [0. 0. 0. 0.]  
 [0. 0. 0. 0.]]
```

```
O =  
[[1. 1. 1. 1.]  
 [1. 1. 1. 1.]]
```

```
D_diag =  
[[1 0 0]  
 [0 2 0]  
 [0 0 7]]
```

```
M =  
[[8 7 3 5 1 9]  
 [2 8 6 3 4 2]  
 [9 4 4 7 7 5]  
 [7 3 2 4 9 3]  
 [4 5 2 4 5 7]  
 [6 2 5 4 5 9]]
```

```
Diagonal entries =  
[8 8 4 4 5 9]
```

```
T =  
[[ 2. -1.  0.  0.  0.  0.]  
 [-1.  2. -1.  0.  0.  0.]  
 [ 0. -1.  2. -1.  0.  0.]  
 [ 0.  0. -1.  2. -1.  0.]  
 [ 0.  0.  0. -1.  2. -1.]  
 [ 0.  0.  0.  0. -1.  2.]]
```